BOUNDARYLAB | Verified Engineering Runbook

Team ODIN - Physical AI CI/CD, robotics falsification, cloud execution and release assurance

Technical audit and implementation playbook | 25 October 2026

- Executive build decision
  - The idea in 30 seconds
  - Architecture and authority boundaries
- Audit of the prior runbook: corrections and omissions
- Gate mechanics: what counts as proof
  - Dependencies / critical path
  - G00 | Freeze rules, scope and owners | Human + Claude | ~1 hour
  - G01 | Local workstation and toolchain | Human installs, Claude verifies
  - G02 | Contracts, repository, CI harness | Claude owns, human reviews
  - G03 | Nebius account, permissions and resource boundary | Human owns
  - G04 | Install NPA and make a genuine Nemotron call | Claude can operate with credential
  - G05 | Real MuJoCo simulation and observable robot control | Simulator engineer
  - G06 | Deterministic specification oracle | Core engineer + simulator engineer
  - G07 | Baselines, seed control and experimental design | Both engineers
  - G08 | Tavily real evidence step | Platform engineer
  - G09 | Cloud GPU workload and NPA workflow | Optional for legal minimum, high-value if accessible
  - G10 | Adversarial optimizer and failure minimizer | Core research engineer
  - G11 | CI/CD release decision and user interface | Platform engineer
  - G12 | Final benchmark, demo, public repo, submission | Both, submission owner signs
- Advanced model path: only AFTER G10 is working
  - G13 | Isaac Lab / Isaac Sim compatibility matrix
  - G14 | GR00T checkpoint and embodiment compatibility
  - G15 | Cosmos / edge-aware risk / cross-model disagreement

- Benchmark math, definitions and judge-proof methods
- Claude Code: exact split of responsibility, handoffs, and prompts
  - Minimal `CLAUDE.md` seed - paste into project root
  - Prompt A | First Claude Code session
  - Prompt B | Cloud and Nemotron verification
  - Prompt C | Robotics search / validation
  - Prompt D | Pre-submission adversarial review
  - Suggested structured gate report
- Security, reliability and observability production checklist
- Day-by-day critical path and stop-loss rules
  - Three-minute storyboard (only recorded actual runs)
- Grounding / anti-hallucination protocol
- Source register (live checked 25 October 2026)

> **Document status: AUDITED BLUEPRINT, NOT AN IMPLEMENTED SYSTEM.** Public documentation was checked on 25 October 2026. All checkboxes begin open. Cloud account permissions, quota, actual model catalog, chosen OS and GPU configuration were not available for testing. A provider command copied from its current documentation is marked **DOC-VERIFIED**; it is not marked **EXECUTED** unless your team runs it and captures evidence. Application source code and JSON contracts are **PROPOSED**. See the anti-hallucination and acceptance protocol at the end.

Executive build decision

Build the robot-policy crash-testing pipeline before building a sophisticated robotics foundation-model demonstration. BoundaryLab should prove: a real controller or policy succeeds on a nominal simulated manipulation task; targeted perturbations reveal a true failure; a minimizer reduces a failing scenario; an immutable artifact bundle makes the failure reproducible; regression evaluation blocks a bad release; and Nebius-hosted NVIDIA model inference contributes validated test hypotheses. That is a believable, research-oriented Physical AI CI/CD product, and a strong demo even without GR00T and Cosmos running [S01][S02].

Hard deadline: Friday **30 Oct 2026, 10:00 AM PDT / 1:00 PM EDT**. Aim to finish the submission on Thursday, October 29. For a simulation-only Physical AI project, the video must show key application modules operating for **at least one minute**; the public YouTube video must be **less than three minutes**; the public code repository must have a visible open-source license and reproducible instructions [S01]. The official requirement is **at least one NVIDIA open-source model and an application that makes a runtime Token Factory call OR runs on Nebius AI Cloud compute**. Using both is desirable but *not a prerequisite to eligibility*. Tavily's separate bonus requires a **functional runtime Tavily API call** [S01].

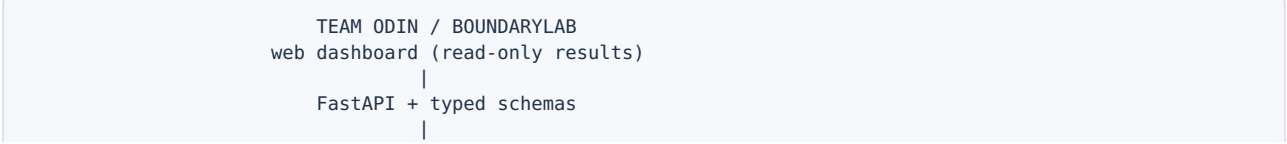
Scope lock: one robot/task; one meaningful observation-to-action loop; 4 perturbations maximum until baseline works; an immutable policy release manifest; nominal/random/targeted search comparison; failure minimization and replay. A dashboard has 4 surfaces only: Test Lab, Failure Atlas, Counterexample Replay, Release Gate.

The idea in 30 seconds

> A robot that succeeds 99 times may still fail if its camera is displaced slightly or its observations arrive late. BoundaryLab is like crash-testing a car: rather than celebrating normal operation, it searches systematically for the **nearest world that breaks a policy**. It captures that counterexample, turns it into a permanent automated regression test, and refuses to declare a new policy release eligible until the required evaluations pass.

Core thesis: Under the same simulation budget and the same allowed parameter domain, active adversarial search will discover failing configurations and/or *closer-to-nominal* failures more effectively than uniform random search. This is a **hypothesis to test, not a promised result**. The distinct system contribution is the *closed-loop engineering workflow* - search -> falsify -> minimize -> archive -> regression -> evidence-backed release decision. Prior adversarial robot testing exists; do not claim to have invented it.

Architecture and authority boundaries





Cloud control-plane distinction: Nebius Token Factory is hosted model inference, **not** a GPU container runtime. NPA/Physical AI Workbench controls configured workloads and artifacts; NPA local installation does **not** automatically grant Nebius cloud quota or create a working SkyPilot/Kubernetes runtime. Serverless Jobs, Endpoints, DevPods and an NPA workflow are related but distinct paths; do not substitute an invented API for one of them [S02][S03][S04].

Audit of the prior runbook: corrections and omissions

Earlier issue	Corrected engineering decision	Why this matters
Implied both Token Factory and Nebius GPU are mandatory	Rules say **Token Factory OR Nebius AI Cloud**, with an NVIDIA open model	Reduces submission risk if GPU quota fails [S01]
Treated a Token Factory catalog check as enough	Must perform generation and inspect **nonempty completion** with model ID	Authentication != inference [S03]
Listed Isaac Lab as a straightforward install	Pin an **Isaac Sim / Isaac Lab / Python / driver / CUDA** compatibility matrix for a chosen tag; docs move	Prevent dependency breakage [S06]
Promoted GR00T before proving action compatibility	Require a real observation -> model -> action -> simulator closed loop; otherwise label as stretch	Loading weights isn't robot control [S07]
'100-1000 rollouts' before establishing economics	Start with 1 -> 10 -> 25 episodes; authorize a scaling budget after per-episode cost	Avoid quota/billing surprises
Suggested MuJoCo and Isaac Lab might share environment	Use separately locked environments or containers, separated by the same adapter contract	Conflicting dependencies
Claimed fully deterministic replay	Require **toleranced replay or statistical reproduction**; preserve seeds, versions and precision info	Parallel/GPU solvers need not be bit-identical
Suggested camera angle / latency / lighting changes for any policy	Only perturb variables *actually observed/used* by that policy; audit coupling	Otherwise experiments are meaningless
'Minimal failure' sounded globally optimal	Report **locally minimized observed counterexample** unless rigorous optimality proven	Scientifically accurate
Published `NEBIUS_TOKEN_FACTORY_BASE_URL` as required	It is an **optional override**, not a required credential variable; use the provider default	Prevent unnecessary configuration and mistakes [S03]
Proposed `DATABASE_URL` password in example	Do not publish real credentials; use placeholder-only `.env.example`; local Compose secrets stay local	Reduce exposure
NPA workflow sample involved Cosmos 3	Workflow validation is useful; **do not launch a heavyweight Cosmos workload** as a first cloud smoke test	One example may require H100 and gated assets [S02]
Missing clean-room independent baseline	Match rollout budgets, random seeds and reporting, include failure to beat baseline	Defensible novelty claim
Weak handling of false-positive PASS	`ERROR` and `INCOMPLETE` are fail-closed; oracle missing telemetry means invalid run	Correct release engineering

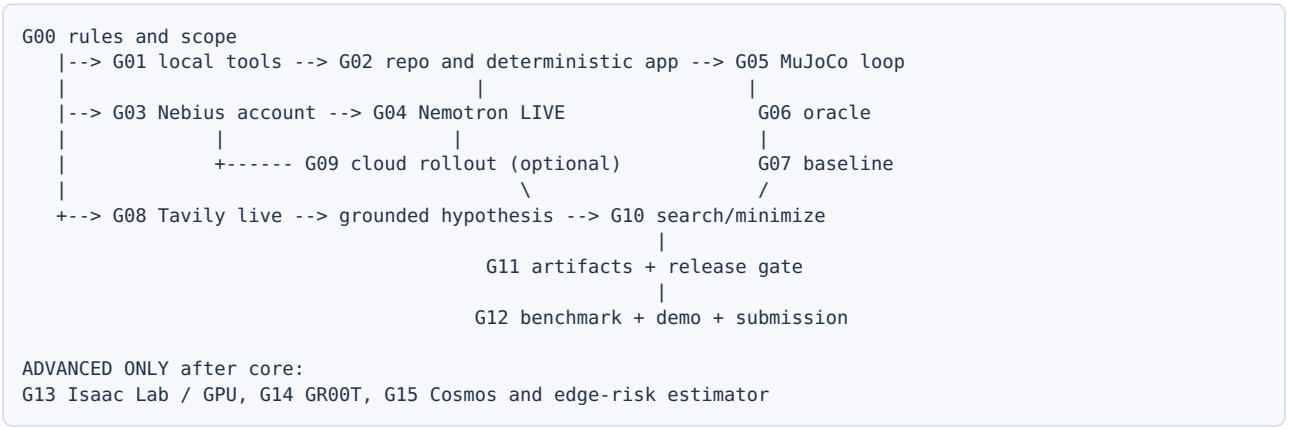
Earlier issue	Corrected engineering decision	Why this matters
Missing human-approved compute cap / teardown evidence	Every paid job needs cost estimate, resource cap, job owner, teardown and post-run cost check	Commercially credible cloud ops
Missing IP, privacy and public repo gate	Check robot asset licenses, model use terms, data consent and public source visibility	Mandatory eligibility [S01]
Prior UI examples risked looking like product screenshots	Any mockup must be explicitly labeled; recorded demo must show functioning modules	Avoid a misleading submission

Gate mechanics: what counts as proof

Each gate has: **preconditions -> operator actions -> machine checks -> preserved artifacts -> pass decision -> fallback**. Each gets a signed record in `docs/gates/G##_report.md`, referencing command logs in `evidence/G##/` (scrub secrets). Allowed statuses: `PASS`, `FAIL`, `BLOCKED`, `NOT_STARTED`. **Do not mark a gate PASS because code exists.** A gate passes only after an actual working environment and required test evidence.

A human approves only known paid actions. Claude Code can operate shell tools when permitted; it cannot invent permissions, determine billing from memory, accept legal terms, or impersonate completed hardware tests. Your first integration choice should be CLI/SDK + files. MCP can come later [S02][S11].

Dependencies / critical path



G00 | Freeze rules, scope and owners | Human + Claude | ~1 hour

01 Read official rules and track requirements yourself [S01]. Write `docs/rules.md`, including Oct 30, 10 AM PDT, physical clip rule, under-three-minute video, public repo/license, NVIDIA open model and Nebius runtime requirement. **02** Confirm team's entrant eligibility, approved team members, ownership and originality. If prior code predates the contest, document significant new work. **03** Register/confirm Devpost and submission owner. **04** Choose provisional project identifier `boundarylab-odin`, track `Physical AI`, and source license compatible with third-party licenses. **05** Assign two ownership lanes: Engineer A - robot simulator, controller, evaluation; Engineer B - orchestration, cloud, Nemotron/Tavily, UI; cross-review on every merged PR. **06** Define MVP: one robot, one stable action space, one physical scene, controlled perturbations, one safety/goal oracle. **07** Budget owner approves ceiling, permitted resource types and teardown procedure. **08** Create `docs/sources.md` with links/versions and a `docs/gates/` directory.

Acceptance: `docs/rules.md` and `docs/scope.md` present, deadline verified, roles/owners fixed, budget cap recorded. **Fallback:** no spending/cluster creation until owner confirms budget. **Evidence:** dated PDF/URL source register and G00 report.

G01 | Local workstation and toolchain | Human installs, Claude verifies

09 Record OS (`uname -a` or Windows/macOS equivalent), architecture, RAM, disk, network proxy and available GPU. **10** Choose canonical workstation: Ubuntu 24.04 LTS or WSL2 Ubuntu for this runbook; macOS team members can build API/UI and some MuJoCo simulation but should not assume Isaac/GR00T GPU compatibility. **11** Install Git, Git LFS, build-essential, `curl`, `jq`, `ffmpeg`, Python with `venv`. On Ubuntu:

```
sudo apt update
sudo apt install -y git git-lfs curl wget jq unzip zip \
  ca-certificates build-essential cmake python3 python3-pip \
  python3-venv ffmpeg
git lfs install
```

12 Verify binaries: `git --version`, `python3 --version`, `git lfs version`, `ffmpeg -version`. **13** Install supported Node LTS from official Node installation instructions (using a version manager is reasonable); record `node --version` and `npm --version`. **14** Install Docker Engine + Compose via Docker's current OS-specific documentation; run `docker compose version` and `docker run --rm hello-world`. Docker group membership is privileged; do not grant broad access casually [S10]. **15** Install Claude Code via its official installation instructions; on Linux/macOS/WSL the currently documented native command is:

```
curl -fsSL https://claude.ai/install.sh | bash
claude --version
```

This command fetches a vendor script; review the vendor and endpoint before execution [S11]. **16** Authenticate Claude Code interactively. **17** Set Claude Code permissions so secret display, installation, cloud write, cloud delete, GPU launch and public publishing require explicit human consent [S12]. **18** Make a workspace under `~/projects/boundarylab-odin/`; `git init`; add `.gitignore` covering `.env`, local secrets, virtualenvs, big checkpoints, user credentials, cloud artifacts and Node build output. **19** Create `.venv`; install app requirements *separately* from MuJoCo/Isaac/NPA. **20** Initialize Next.js TypeScript frontend only after minimal API test passes. **21** Record lockfiles and versions, not just `pip install` commands. **22** Run network and filesystem smoke checks, including ability to open official repository URLs.

Acceptance: Claude CLI opens; `python -m pytest` can execute a smoke test; Docker (if chosen) starts; Git commit exists; `GET /health` returns 200 and UI can retrieve it. **Evidence:** `evidence/G01/{versions.txt,tests.txt,health.json}`. **Fallback:** if Docker unavailable, run FastAPI natively; do not block all simulation research on Compose.

Minimal app bootstrap commands [PROPOSED app dependencies]

```
mkdir -p ~/projects/boundarylab-odin && cd ~/projects/boundarylab-odin
git init
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install fastapi 'uvicorn[standard]' pydantic \
  httpx numpy scipy pytest pytest-asyncio ruff python-dotenv \
  tavily-python openai
python -m pip freeze > requirements.initial.txt
```

`requirements.initial.txt` is a capture file, not a complete reproducible lock strategy; produce a pinned lock after dependency resolution. Keep Postgres, Polars, MLflow, LangGraph, React charting and all GPU libraries out until an actual feature needs them.

G02 | Contracts, repository, CI harness | Claude owns, human reviews

23 Write `CLAUDE.md`: *never invent provider flags; read `--help`; real execution required before PASS; no expense or secrets without approval; deterministic oracle owns release decisions*. **24** Create an app-level monorepo with `api/`, `core/`, `sim/`, `search/`, `oracle/`, `artifacts/`, `release/`, `web/`, `tests/`, `docs/`, `evidence/`, `.github/workflows/`. **25** Define `Scenario`, `Observation`, `Action`, `Trajectory`, `EpisodeResult`, `PolicyManifest`, `ReleaseDecision` schemas using `pydantic` with `extra='forbid'`. **26** Define SI units and ranges for each scenario variable and version the schemas. **27** Define simulation adapter interface (`reset`, `observe`, `act`, `step`, `terminate`, `save_trajectory`) with explicit errors on unsupported

perturbations. **28** Implement artifact manifest schema and SHA256 calculation. **29** Add Ruff/pytest; require tests with one passing case, one collision failure, malformed data rejection. **30** Add minimal GitHub Actions workflow invoking linters and unit tests; no secrets and no cloud spend on every PR.

Acceptance: schemas reject unknown keys and invalid ranges, unit tests pass with fake fixtures, CI passes in public/private repo as appropriate. **Evidence:** commit SHA, CI URL, test log. **Fallback:** use SQLite + filesystem artifacts for MVP; Postgres is optional until concurrency demands it.

G03 | Nebius account, permissions and resource boundary | Human owns

31 Create/confirm Nebius Cloud account, tenant, project, region and approved role. **32** Check whether hackathon Builder Program benefits exist for your account; never invent a credit amount or transferability [S01]. **33** Record observed compute/Token Factory prices, quotas and any actual grant balance (screenshots may contain private data; redact before sharing). **34** Create separate Token Factory credential and Cloud IAM/CLI identity; they are **not interchangeable** [S02][S03]. **35** Select budget cap, max concurrent jobs (start 1), resource allowlist, maximum experiment count and alarm/check-in procedure. **36** Create/authorize an artifact bucket only if the chosen execution method requires it, and define private storage retention. **37** Document regions, bucket endpoint, credentials resolution and teardown/cleanup responsibilities. **38** Require human confirmation before `npa configure` if default interactive flow would provision object storage.

Acceptance: identity/project documented, approved cost ceiling, keys stored securely outside repo, credentials not leaked, workflow path chosen. **Evidence:** redacted account check and signed budget note. **Fallback:** run live Token Factory + local MuJoCo for eligibility, while cloud simulation remains stretch.

G04 | Install NPA and make a genuine Nemotron call | Claude can operate with credential

The public `nebius-physical-ai` repo says `npa` installs from the checkout, not PyPI; local `npa --version` works without cloud credentials. Do not clone the evolving `main` branch and later forget its commit [S02].

39 Clone Workbench adjacent to the app repo. **40** Record commit hash. **41** Make dedicated Python environment and editable install. **42** Verify `npa --version`, `npa workbench --help`. **43** Install and configure Nebius CLI using the **specific compatible instructions in that checked-out quickstart**; do not hardcode an installer version lifted from another date. **44** Use `npa configure --no-provision` initially if storage is not yet approved. **45** Run Nebius health preflight. **46** Enter Token Factory key in protected environment or NPA credentials file with file mode `0600`. **47** Execute `status`, `verify`, `models` and choose an **available NVIDIA Nemotron** model. **48** Run actual `generate` with a typed prompt file. **49** Parse response and require a nonempty `completion`; preserve ID/model/date/runtime, excluding secrets. **50** Validate model hypothesis JSON into allowlisted physical variables; reject ungrounded API/model claims.

```
cd ~/projects
git clone https://github.com/nebius/nebius-physical-ai.git
cd nebius-physical-ai
git rev-parse HEAD
python3 -m venv .venv
source .venv/bin/activate
pip install -e npa
npa --version
npa workbench --help
# After installing/authenticating the Nebius CLI as in pinned quickstart:
npa configure --no-provision
npa configure --show
npa workbench health preflight --checks nebius --json
npa workbench token-factory status
npa workbench token-factory verify
npa workbench token-factory models
```

Then only after catalog lookup:

```
mkdir -p token-factory-results
cat > token-factory-results/prompts.jsonl <<'EOF'
{"id":"test-001","prompt":"Propose two falsifiable bounded perturbation hypotheses for a robot block
        placement task. Output a concise explanation only; do not decide safety."}
EOF

TEXT_MODEL='<COPY_ACCESSIBLE_NVIDIA_MODEL_ID_FROM_CATALOG>'
npa workbench token-factory generate \
 --input-path token-factory-results/prompts.jsonl \
 --output-path token-factory-results/generations.jsonl \
 --model "$TEXT_MODEL" --output json
```

DOC-VERIFIED: the current NPA docs show `npa workbench token-factory generate --input-path ... --output-path ... --model ... --output json`. They list `nvidia/Nemotron-3_5-Lightning` as an example default, **not guaranteed available**. The default Token Factory API base URL is `https://api.tokenfactory.nebius.com/v1/`, with `NEBIUS_TOKEN_FACTORY_BASE_URL` only if overriding [S03].

Acceptance: one authenticated live completion from a listed NVIDIA model, captured to JSONL and parsed; not simply `model_count > 0`. **Evidence:** redacted JSONL, catalog model ID, CLI logs. **Fallback:** if the model is absent, choose another actual NVIDIA open model from the available catalog; do not fabricate an ID. If none is available, G04 is BLOCKED and contest compliance is in danger.

G05 | Real MuJoCo simulation and observable robot control | Simulator engineer

Google DeepMind MuJoCo documents `pip install mujoco` and includes its own native library; you do not separately download the binary unless needed for another workflow [S05]. Use the model collection in MuJoCo Menagerie when license and task fit [S08].

51 Create `sim-mujoco` Python environment or container independent of NPA/app. **52** `pip install mujoco` then import and report `mujoco.__version__`. **53** Select a physically meaningful task **you can actually execute this week**. Prefer pushing a block into a region, or a pick-place task *only if* the controller/grasp is demonstrated; a push task may be a safer early fallback. **54** Import a compatible model or build a small XML scene, record asset licenses and scene file hash. **55** Implement a feedback controller whose action updates from current observation each timestep, not a rendered prerecorded trajectory. **56** Prove an actual action changes joint/object states and physics advances. **57** Log states at fixed simulator-step cadence and produce short video via the simulator renderer. **58** Test an obviously failing scenario. **59** Repeat same seed, initial state and software version to test reproduction; compare trajectories within tolerances. **60** Implement at least **two real perturbations** (object pose/mass/friction; observation delay only if observations are used). Add a test asserting that each changes simulator parameters or policy-observed data. **61** Expand to a maximum of 4 perturbations only after the first two are effective. **62** Record latency semantics: an N-ms delay comes from an observation history buffer indexed by simulator time; account for controller frequency and discretization.

```
python3 -m venv .venv-mujoco
source .venv-mujoco/bin/activate
python -m pip install --upgrade pip
python -m pip install mujoco numpy pytest
python -c 'import mujoco; print("MuJoCo", mujoco.__version__)'
```

Acceptance: nominal episode, real failed episode, per-step trajectory, truthful video, and meaningful causal perturbation. **Evidence:** scenario JSON, trajectory, output video, simulator version, policy hash, checksums. **Fallback:** push/reach before complex grasp; NEVER quietly label a fixed scripted replay as a learned robot policy.

G06 | Deterministic specification oracle | Core engineer + simulator engineer

63 Define task completion through simulator state: object within target radius, stability for N steps, deadline, collision rules and workspace limitations. **64** Define invalid termination and missing telemetry. **65** Separate task failure from **safety violation**; an object missing the bin is not necessarily a collision hazard. **66** Ensure safety conditions measure actual available sensors. Do not report force limits if force telemetry is absent or uncalibrated. **67** Implement episode state machine: `RUNNING`, `PASS`, `TASK_FAIL`, `SAFETY_FAIL`, `INVALID`, `TIMEOUT`, `EXEC_ERROR`. **68** Test exact known fixtures for each status. **69** Confirm LLM-supplied values cannot override thresholds; only a versioned approved `TestSpec` can do that. **70** Treat NaN, missing field, timebase mismatch, impossible physical values and incomplete artifact as INVALID. **71** Add optional quantitative robustness score (distance to goal, collision clearance, deadline slack), but only if derivation is grounded in measured state.

Acceptance: oracle catches known violations, does not hallucinate missing force data, distinguishes bad policy from infrastructure problems, and reliably blocks invalid results. **Evidence:** oracle unit tests, sensitivity tests, spec SHA256.

G07 | Baselines, seed control and experimental design | Both engineers

72 Lock benchmark task, base scenario, perturbation domains and policy version **before** evaluating search strategies. **73** Partition training/search seeds and held-out evaluation seeds; also reserve held-out parameter configurations, not just different random-number seeds. **74** Run 3-10 nominal episodes to verify whether the policy is worth testing. **75** Set a first evaluation budget (example: 25 random / 25 active episodes after warmup) based on actual measured throughput, not an arbitrary promise of 1,000 cloud rollouts. **76** Fix the random generator and record candidate vectors. **77** Store all episodes, including failures, invalids, retries and resumed jobs. **78** Choose a normalized distance to nominal and publish variable scaling/weights. **79** Define primary metrics before looking at results: episodes-to-first-failure, closest failing distance at same rollout budget, failure reproduction rate, elapsed time/cost. **80** Run random search baseline and verify evaluator agreement by manually replaying a few cases. **81** Record variance across independent search repetitions where feasible.

Acceptance: a genuine random baseline exists with auditable candidate vectors and all episode results; no experiment double-counting. **Evidence:** locked protocol, seed lists, CSV/Parquet + checksums.

G08 | Tavily real evidence step | Platform engineer

82 Set Tavily key in private environment; check official SDK and request budget [S09]. **83** Make one live `TavilyClient.search(...)` call from a script and preserve timestamp, response status and result URLs (no secrets). **84** Search actual manufacturer/simulator documentation relevant to the exact chosen embodiment. **85** Separate sourced facts from model proposals. **86** Create a `DocumentedConstraint` object with source URL, units, quote/paraphrase, extraction confidence, authorizer and approval state. **87** Reject source text that tries to command the coding agent or raise safety limits - the fetched page is untrusted evidence. **88** Add a user-visible approval step before modifying the authoritative simulator parameter bounds. **89** Pass approved facts + existing failures to Nemotron for a *hypothesis*; validate generated parameters against approved enums/ranges. **90** Link proposed hypothesis to the actual scenario and subsequent result.

```python
# scripts/smoke_tavily.py  | DOC-VERIFIED SDK pattern
import os
from tavily import TavilyClient
client = TavilyClient(api_key=os.environ["TAVILY_API_KEY"])
result = client.search(
    "official robot gripper manufacturer payload specification",
    search_depth="basic", max_results=3)
assert result.get("results"), "No usable Tavily results"
```

```
for item in result["results"]:
    print(item.get("title"), item.get("url"))
```

Acceptance: at least one real Tavily request and one linked evidence -> approved hypothesis -> executed scenario. **Evidence:** filtered search output, provenance linkage and human approval event. **Fallback:** if Tavily integration fails, do not claim bonus-award eligibility; other project functionality may continue.

G09 | Cloud GPU workload and NPA workflow | Optional for legal minimum, high-value if accessible

91 Decide whether cloud simulation is justified: if CPU MuJoCo handles 50 rollouts quickly, a GPU cluster is not necessarily necessary. Cloud jobs add value when parallel physics, rendering or GR00T really needs them. **92** Check pricing/quota and account region; get human approval for an exact one-job trial and teardown. **93** Confirm NPA dependencies and selected cloud runtime (managed Kubernetes/SkyPilot or documented alternative) as described by checked-out version [S02][S04]. **94** Select a small existing NPA workflow and run `validate-spec`, `plan-spec` without launches. *The published `cosmos3-generate.yaml` is a validation example, not a cheap smoke workload.* **95** Build custom runtime that accepts a typed scenario and writes manifest/trajectory/result to a writable artifact location. **96** Confirm container digest, working directory, input and artifact paths in local container. **97** Create an NPA specification by copying the **actual version's** documented schema/toolRef; do not invent YAML keys. **98** Perform image preflight and verify note: it may make temporary probe pods and leave a service account, so it is not guaranteed side-effect-free [S04]. **99** Submit **one** approved real run, inspect job status/logs, retrieve manifest and video, verify checksum. **100** Tear down or confirm auto-release of cloud resources and inspect cost. **101** Scale to 5 then 20 cloud episodes only after per-episode resource measurement and complete artifact success.

Relevant vendor commands to inspect before use (DOC-VERIFIED [S04]):

```
npa workbench workflow validate-spec --help
npa workbench workflow plan-spec --help
npa workbench workflow prepare-run --help
npa workbench workflow preflight-images --help
npa workbench workflow submit --help
npa workbench workflow status --help
npa workbench workflow logs --help
npa workbench workflow artifacts --help
npa workbench workflow cancel --help
```

The published NPA guide uses `npa workbench workflow submit "$workflow_spec" --project "$project_alias" --infra "k8s/$cluster_name" --run-id "$run_id" --var "bucket=$bucket_name" --runtime` for its workflow example [S04]. **Do not copy and run without the matched workflow spec, cluster, bucket, model tokens and active quota.**

Acceptance: actual Nebius run ID, authorized compute allocation, correct policy trajectory artifacts, checksum verification, teardown proof and cost record. **Fallback:** Token Factory-backed model inference + local MuJoCo satisfies the quoted *either-or* cloud requirement; describe simulation truthfully as local [S01].

G10 | Adversarial optimizer and failure minimizer | Core research engineer

102 Implement `Objective(x)`: run policy against scenario x and compute `task_fail`, `safety_fail`, `distance_from_nominal` and optional graded violation. **103** Start with a **simple bounded adaptive search** (e.g., differential evolution or a transparent surrogate/random proposal policy); make the algorithm inspectable and reproducible. **104** Use identical allowed ranges and equal **number of simulator episodes** for random and active search. Count retries/failed jobs separately. **105** Log every candidate and stage. **106** Identify a failing scenario; if none appears, widen only the *declared* test domain and restart fair baselines. **107** Counterexample minimization: greedily revert one changed

variable to nominal; if failure persists, keep the revert. **108** Refine numeric values with a bounded local search; test nonmonotonicity and never assume binary search universally works. **109** Repeat the minimized candidate on a controlled seed set; report observed failure probability with an interval if multiple runs. **110** Persist both original and minimized scenarios and real replay footage. **111** Do not call the result global minimum or causal proof; call it 'a locally minimized, reproducible failure under our tested simulator and controller.'

Example minimization pseudocode [PROPOSED, not executable vendor API]

```
def minimize_failure(initial, nominal, run, repeats=3):
    best = initial.copy()
    for key in sorted(best, key=lambda k: abs(best[k] - nominal[k])):
        candidate = best.copy()
        candidate[key] = nominal[key]
        outcomes = [run(candidate, seed=i) for i in range(repeats)]
        # Define a reproducibility threshold before observing outcomes.
        if sum(o.is_valid_failure for o in outcomes) >= repeats:
            best = candidate
    return best  # locally reduced; no claim of global optimum
```

Acceptance: documented search baseline and repeatable minimized failure with smaller normalized perturbation distance and stable artifact URIs. **Evidence:** candidate ledger, comparison plot, replay videos, run manifests.

G11 | CI/CD release decision and user interface | Platform engineer

112 Add policy registry with immutable checkpoint hash + model description (learned, classical or heuristic); don't mislabel a classical controller as GR00T. **113** Add scenario/test registry with versioned validation limits. **114** Implement release candidate manifest with exact policy, simulator, test-suite, seed lists and commit hash. **115** Implement status machine: `ELIGIBLE`, `BLOCKED`, `INCOMPLETE`, `ERROR`. **116** Fail closed when telemetry missing, job status unknown, required regression not evaluated, artifact integrity mismatch or mismatch between checkpoint and test results. **117** Make nominal, saved-regression and independent held-out cases mandatory for release eligibility. **118** For a repair demo, change something concrete (e.g., control delay compensation or feedback control tuning), re-run complete suites and admit if no improvement occurred. **119** Add workflow that executes unit tests on GitHub Actions and Cloud/simulator evaluation only when authorized (PRs should not create arbitrary paid GPU clusters). **120** Build four dashboard modules: Test Lab, Failure Atlas, Counterexample Replay, Release Gate; mark mock/demo-only screens. **121** Wire dashboard to actual run records and replay videos. **122** Validate release test with deliberate missing log, corrupt artifact, failed cloud job and known safety violation.

Acceptance: an approved policy version with passing required tests shows `ELIGIBLE`; each fault injection reliably yields `BLOCKED`, `INCOMPLETE` or `ERROR`; front-end does not display green from mock data. **Evidence:** CI URL, release JSON, fault-injection tests, screen recording.

G12 | Final benchmark, demo, public repo, submission | Both, submission owner signs

123 Freeze code; rerun nominal/random/targeted with a consistent policy and recorded CPU/GPU budget. **124** Run held-out tests; separate them from the search optimizer's seen configurations. **125** Record actual baseline table, with failures/episodes, first-failure cost, min distance and cloud usage, including negative findings. **126** Capture **at least 60 seconds** of genuine working simulation/UI modules (the entire short video is <3 minutes), not only narration and slide graphics [S01]. **127** Show nominal success -> live/adversarial testing -> real failure -> reduced counterexample -> replay -> release decision. **128** Show actual Nebius/NVIDIA/Tavily traces only for integrations which genuinely ran. **129** Provide a public open-source repo with root LICENSE, README, exact versions, scripts, dataset/asset licenses, environment variables, smoke tests, platform feedback and limitations. **130** Provide YouTube public URL,

project description, selected Physical AI track and submit early. **131** Independently view submitted Devpost project and video in an incognito session; verify repo link and license. **132** Archive the final evidence manifest and note approximate billed/credited resource use.

Acceptance: public submission passes self-audit, every claimed integration has evidence, no secret is public, all working modules correspond to actual code. **Evidence:** Devpost URL, YouTube URL, repo URL/ commit, test summary and signed G12 report.

Advanced model path: only AFTER G10 is working

G13 | Isaac Lab / Isaac Sim compatibility matrix

Prerequisites: real free GPU capacity, host driver compatibility and an already working core project. Isaac Lab docs have changed location; choose a fixed Isaac Lab git tag and follow that tag's matching Isaac Sim / Python / CUDA/container instructions [S06]. Check GPU/VRAM and headless rendering separately. Prove an upstream Isaac tutorial runs in the exact GPU environment. Implement `IsaacAdapter` and the **same scenario/trajectory contract**; do not make changing simulator invalidate all core tests. Export a simulated rollout and compare physics semantics (friction, observation latency and collision telemetry) carefully. Cross-simulator agreement is *not* automatic.

Acceptance: one documented NVIDIA simulator tutorial + one BoundaryLab policy rollout + artifact proof. **Stop-loss:** if environment setup consumes more than half a workday without upstream smoke passing, pause and preserve MuJoCo MVP.

G14 | GR00T checkpoint and embodiment compatibility

Use official [S07]. Check required weights/terms/hardware; use separately pinned GPU environment. Verify official inference script first, then observation transforms, camera/resolution normalization, action dimensions, control frequency, physical joint limits, coordinate frames, gripper conventions, and latency. Only call it *GR00T under test* when the policy receives actual simulator observations and its returned actions drive simulation over time. An inference-only generated action is not a closed-loop robot demonstration. Fine-tuning is optional; no promise of 40 GB GPU availability or universal action compatibility. If architecture clashes, do not contaminate the core app environment.

G15 | Cosmos / edge-aware risk / cross-model disagreement

Research extension: have the simulator and a learned world model independently predict an aligned future, then investigate disagreement with grounded physical evaluation. This requires camera alignment, state projection, evaluation protocols, model capacity and systematic physical-consistency checks; do not call generated video a safety oracle. Optional edge risk estimator estimates imminent failure risk under a calibrated, held-out scenario distribution and triggers a deterministic fallback; it is not a certification claim [S13].

Benchmark math, definitions and judge-proof methods

Parameter vector: `x = [object_offset_m, object_mass_kg, friction_coefficient, observation_delay_ms]`. Choose only variables the actual policy/simulator exposes. **Nominal x0** is approved before testing. **Distance** should normalize units: `D(x,x0) = sqrt(sum_i w_i * ((x_i-x0_i)/range_i)^2)`. Weights and bounds cannot be tuned after seeing results without disclosing it.

Observed counterexample: scenario and seed for which the real executed trajectory violates approved specifications. **Replay rate**: number of confirmed failures / valid repeated runs on the designated repeated seed set. **Minimum observed failure margin**: smallest normalized `D` among *observed* failing worlds; this is not a universal safety margin. **Regressions**: counterexamples discovered on version `v1` that form a fixed required suite for `v2`. **Held-out test**: new configurations/seeds not used during optimization or repair. **Runtime robustness score**: optional temporal-logic margin; specify mathematical semantics and sensor units before reporting.

Quantity	Must be recorded	Failure mode to avoid
Success/task completion	Goal state, deadline and source telemetry	Visual judgment from one frame
Safety violation	Collision/force limits from real simulator fields	Unsupported invented ‘force’ value
Efficiency	Episodes to first verified failure; wall-clock	Comparing unequal episode budgets
Nearest failure	Metric definition and actual perturbed parameters	Comparing incompatible units
Repeatability	Repeats, seed set, failure frequency	One-off stochastic event called deterministic
Improvement	`v1` vs `v2`, original and held-out tests	Optimizing only the known one-off failure
Cost	Actual time, GPU class, billed usage or approved estimates	Unverified claims of free cloud compute
Provenance	Policy checkpoint hash, scenario hash, runtime/image digest	A file called `latest.pt` treated as immutable

Suggested first experiments: `N_nominal=5`, `N_random=25`, `N_active=25`, with matched budgets. Expand only after throughput/cost measured; report tiny sample size as a limitation. For a stronger study repeat the search with multiple independent random seeds and use confidence intervals, not cherry-picked best runs. Evaluate number of failures and min distance versus cumulative episodes as **curves**, not one flattering cherry-picked point.

Release gate: run mandatory nominal and regression cases; treat all unexecuted or missing cases as `INCOMPLETE`; any verified violation yields `BLOCKED`; job failure/corrupt evidence yields `ERROR`; only all-required-passed yields `ELIGIBLE`. The word eligible means **passing a named simulation suite**, not safe for a real robot. Human approval is additionally needed for hardware deployment.

Claude Code: exact split of responsibility, handoffs, and prompts

Correct mental model: Claude Code can install packages, run terminal commands and operate authenticated provider CLIs **in the environment in which it is running**, subject to permissions. It cannot reach your laptop's keys if you're running Claude in a separate remote environment without provisioning access. It doesn't possess Nebius/NVIDIA GPU quota just because it can generate code. Be explicit which shell is used. The official NPA repo specifically describes coding agents controlling its CLI [S02].

Task	Claude can automate?	Human approval/verification
Read official docs and installed `--help`	Yes	Cross-check versions/links
Write API, schemas, tests, Dockerfiles	Yes	Review interfaces and security
Install local packages	Yes, authorized terminal	Review potentially privileged commands
Create cloud keys/accept licensed model terms	Not independently authorized	Owner does this
Call live model/Tavily APIs	Yes, when configured	Supply keys securely, cap usage
Launch NPA/GPU jobs	Yes technically	Approve exact plan and spend
Debug simulator and evaluate results	Yes	Inspect videos and raw trajectories
Declare robot 'certified safe'	No	Not supported by simulation tests
Submit Devpost and public YouTube	Prepare assets/links	Owner completes, confirms public visibility

Minimal `CLAUDE.md` seed - paste into project root

```
# BoundaryLab / Team ODIN
MISSION: falsification-driven CI/CD for embodied robot policies.
CRITICAL PATH: working closed-loop simulator -> deterministic oracle ->
random baseline -> active search -> minimal reproducible counterexample ->
regression gate -> verified sponsor integrations -> demo.

Never invent provider APIs, models, credit amounts, quotas or performance.
Read the pinned vendor docs and installed `--help` before any provider call.
Secrets remain outside repository; paid provisioning requires written approval.
Only real runtime calls count as integrations. Keep full artifact provenance.
Do not let an LLM override test constraints or report pass/fail.
Do not claim a controller is GR00T or learned unless integration proves it.
Prioritize G00-G12; G13-G15 are explicitly optional.

At each gate write: prerequisites, source URLs/versions, commands run,
actual outputs, artifacts, test evidence, cost/approval, PASS/FAIL/BLOCKED,
blockers, and next gate. Stop on unverified critical prerequisites.
```

Prompt A | First Claude Code session

```
Act as the engineering lead for BoundaryLab. Read CLAUDE.md and this
verified runbook. Start with G00-G02, not GR00T/Cosmos.
1. Inspect the actual operating system and installed tools.
2. Produce a requirements/compatibility inventory.
3. Create typed app schemas, project scaffold, tests and CI baseline.
```

```
4. Run tests, save raw output and explain every failure.
5. Write docs/gates/G00_report.md through G02_report.md.
Do not report PASS without execution evidence; never launch paid cloud compute.
```

Prompt B | Cloud and Nemotron verification

```
Inspect the checked-out nebius-physical-ai commit and its quickstart,
Token Factory docs and npa CLI --help. Do not infer command flags.
Run non-provisioning local NPA checks. Report required human credentials.
When approved, run status/verify/models, select an actual accessible
NVIDIA Nemotron ID and perform a real generate call. Verify a nonempty
completion, redact credentials, save model ID and output as evidence.
Do not provision storage or GPU resources unless explicitly approved.
```

Prompt C | Robotics search / validation

```
Use the MuJoCo scene and policy that actually pass G05. Implement
physically effective bounded perturbations, deterministic oracle,
random baseline and bounded adaptive search. Define scientific metrics
before the test. Record EVERY candidate, seed, policy hash and status.
Find a real failure, minimize it without assuming monotonicity, reproduce
it and generate a replay. Report negative findings when present.
```

Prompt D | Pre-submission adversarial review

```
Act as a skeptical principal robotics validation engineer. Audit:
1) Does policy action truly control physics each timestep?
2) Do perturbations actually affect observations/dynamics?
3) Are all pass/fail outcomes simulator-derived?
4) Is random baseline fairly budgeted?
5) Can the minimized failure be replayed?
6) Are held-out tests uncontaminated?
7) Are Nemotron/Nebius/Tavily claims proven by runtime artifacts?
8) Are failed cloud jobs fail-closed?
9) Are keys, licenses and public assets safe?
10) Does submission video honestly show functioning modules?
Output blocking findings and exact remediation before any grand claims.
```

Suggested structured gate report

```yaml
# Project-owned manifest - not a Nebius API format
schema_version: boundarylab.gate-report.v1
gate: G05
status: BLOCKED  # PASS | FAIL | BLOCKED | NOT_STARTED
source_revision: '<git hash>'
environment: '<host + Python + simulator versions>'
commands: []
evidence_paths: []
provider_run_ids: []
actual_test_results: []
spend_approved: false
approved_by: null
blockers: []
next_action: '<concrete follow-up>'
```

Security, reliability and observability production checklist

Identity: separate work accounts for Nebius/Tavily/NVIDIA; never paste API keys into chat; private environment variables/credential files; least privilege; MFA; rotate exposed secrets. **Isolation:** no untrusted web page can modify test constraints; no arbitrary untrusted tool execution based on model output; validate Pydantic schemas. **Cloud:** approval before paid resources; record region, quota and billing; idempotent job identifiers; explicit cancel/cleanup; bounded retries and timeouts; verify result before treating `succeeded` status as scientific success. **Artifacts:** atomic upload, manifest checksums, duplicate detection, immutable names, retention policy; separate raw and derived artifacts. **Observability:** correlation ID from HTTP request -> hypothesis -> scenario -> cloud run -> trajectory -> oracle result -> UI. **Reproducibility:** lock dependency/driver/container versions, log seeds and precision, disclose nondeterminism, capture hardware type. **Legal:** don’t publish datasets, manuals, generated media, robot models or weights without verifying license and attribution. **User experience:** truthful demo labels, cost visibility, error messages and retry/cancel controls.

Fault-injection tests (mandatory before release): timeout, model/API 401, Token Factory catalog contains no NVIDIA model, Tavily API 429, corrupt trajectory, missing timestamps, NaN state, run status says success but artifact is absent, duplicate job ID, stale policy hash, failed checksum, seed mismatch, simulator scenario supports friction but actually ignores input. All must fail closed or explicitly block the affected feature.

Day-by-day critical path and stop-loss rules

Day (EDT)	A: simulation/research	B: cloud/platform/ product	Hard artifact
Sun Oct 25	MuJoCo controller; one nominal and failed rollout	G00-G04, live Nemotron, typed API/contracts	**Two real simulator videos + live Nemotron JSONL**
Mon Oct 26	Oracle + bounded perturbations + 25 random episodes	Tavily provenance; NPA/ cloud trial only if approved	**Random baseline and typed failure artifacts**
Tue Oct 27	Active search, minimizer, reproduction	Experiment board and run ledger	**Nearest observed failure + replay + comparison**
Wed Oct 28	Validate held-out cases, one explicit repair/fallback	Release state machine, API/ UI and GitHub CI	**Fail-closed release manifest + live dashboard**
Thu Oct 29	Freeze tests/results, record video	README/license, public repo, YouTube, submit	**Valid public Devpost submission**
Fri Oct 30 AM	Emergency correction only	Confirm all links accessible	**Deadline: 1 PM Eastern**

Stop losses: If GPU/Isaac deployment fails after half a day of documented effort, continue CPU MuJoCo while preserving Token Factory integration. If grasp controller is not reliable after half a day, use simple pushing/reaching task rather than fake the success. If adaptive search shows no advantage, report honestly and emphasize minimization/reproducibility as the demonstrated value. If repair fails, show `BLOCKED`; no magical green demo. If a model's terms/access are missing, do not pretend it ran. No additional models before a genuine full loop.

Three-minute storyboard (only recorded actual runs)

Time	Screen/module	One-line narration
0:00-0:15	Nominal robot success	'It passes ordinary tests. Would you ship it?'
0:15-0:40	Actual scenario search and job ledger	'We perturb only physically meaningful variables.'
0:40-1:05	Replayed real failure and oracle	'It breaks under a small, concrete change.'
1:05-1:30	Minimization and reduced replay	'We remove irrelevant changes and find a smaller counterexample.'
1:30-2:00	Tavily source / Nemotron hypothesis / test contract	'The AI suggests; deterministic physics decides.'
2:00-2:25	Release gate / regression rerun	'A bad release gets blocked; a changed policy must retest.'
2:25-2:45	Baseline and artifact proof	'Here are actual results versus random testing.'
2:45-2:58	Architecture and value	'BoundaryLab: crash-test robot intelligence before reality does.'

Video: Keep video **under 3:00**, not exactly 3:00. At least a **continuous combined minute of demonstrably operating application modules** is a safe interpretation of the rule; don’t fill those shots with only a title slide or illustrations. No fabricated speedups or results [S01].

Grounding / anti-hallucination protocol

1. **Source first.** Inspect an official URL and pin its revision. Query `--help` for the checked-out CLI before issuing commands. A model's recollection is not a valid source for a provider flag.
2. **Label epistemic status.** `DOC-VERIFIED` means documentation was seen; `EXECUTED` means your environment ran the command and saved evidence; `PROPOSED` is your own design; `UNKNOWN` requires account-specific verification.
3. **Every integration has a smoke test.** Nebius catalog list != inference; NPA local CLI != cloud compute; container job success != valid robot evaluation; Tavily SDK import != authenticated search; GR00T model loading != closed-loop robot control.
4. **No fake numbers.** Credits, pricing, GPU quota, model IDs and experiment benchmarks must come from account screens or actual reproducible runs. No illustrative demo metric may be published as observed.
5. **Every release is evidence-bound.** Store scenario hash, policy hash, simulator/container versions, seeds, telemetry, model ID, true run ID and evaluator output; missing fields -> `INCOMPLETE` / `ERROR`.
6. **The simulator isn't reality.** Never claim universal safety or formal certification from a finite simulation suite; state the tested envelope and limits.
7. **Threat-model inputs.** Manuals and search results are untrusted text; constrain extraction and require approval of safety limits. No prompt injection into cloud operations.
8. **Audit before submission.** Match all video narration to archived executable evidence, validate README from a clean checkout and inspect the public repository for leaked secrets.

Source register (live checked 25 October 2026)

Source symbols used inline above link to these original, official resources. Repository `main` can change after retrieval; clone and pin your checkout's SHA before using commands. An older installed commit may not have the newly documented tools.

- **[S01] Devpost official hackathon rules** - https://nebiusglobalaihackathon.devpost.com/rules - deadline, eligibility, one NVIDIA open model and *Token Factory or Nebius Cloud*, demo, public license, judging, Tavily bonus.
- **[S02] Nebius Physical AI Workbench README + Quickstart** - https://github.com/nebius/nebius-physical-ai ; https://github.com/nebius/nebius-physical-ai/blob/main/docs/quickstart.md - NPA install, agent CLI, auth, storage behavior, workload prerequisites.
- **[S03] Nebius Workbench Token Factory CLI docs** - https://github.com/nebius/nebius-physical-ai/blob/main/docs/workbench/token-factory.md - status/verify/models, actual generate examples, default URL and current example model IDs.
- **[S04] NPA workflow guide and CLI** - https://github.com/nebius/nebius-physical-ai/blob/main/docs/workbench/npa-workflow-guide.md ; https://github.com/nebius/nebius-physical-ai/blob/main/docs/cli/workflow.md - validate, plan, preflight, submit, status, logs, artifact handling and warnings.
- **[S05] MuJoCo Python bindings** - https://mujoco.readthedocs.io/en/stable/python.html - package installation, simulator stepping, rendering and API semantics.
- **[S06] Official Isaac Lab repository** - https://github.com/isaac-sim/IsaacLab - version-pinned installation guides; prior `/source/setup/installation/` docs have moved.
- **[S07] NVIDIA Isaac GR00T repository and deployment guide** - https://github.com/NVIDIA/Isaac-GR00T ; https://github.com/NVIDIA/Isaac-GR00T/blob/main/scripts/deployment/README.md - actual model workflow, supported tasks and hardware compatibility.
- **[S08] MuJoCo Menagerie official robot models** - https://github.com/google-deepmind/mujoco_menagerie - selectable physical robot assets and their specific licenses.
- **[S09] Tavily Search API / Python SDK** - https://docs.tavily.com/documentation/api-reference/endpoint/search ; https://github.com/tavily-ai/tavily-python - live search invocation and SDK.
- **[S10] Official Docker Ubuntu setup** - https://docs.docker.com/engine/install/ubuntu/ - supported OS, packages, networking/security.
- **[S11] Claude Code Quickstart** - https://code.claude.com/docs/en/quickstart - supported installation and usage.
- **[S12] Claude Code permissions** - https://code.claude.com/docs/en/permissions - explicit approval/security configuration.
- **[S13] NVIDIA Cosmos repository** - https://github.com/NVIDIA/cosmos - world-model tools, hardware prerequisites and limitations; optional research extension.

Revision note: This runbook is a checked, expanded successor to the earlier BoundaryLab step-by-step response and the attached BoundaryLab concept PDF. Exact provider command outputs and your team's implementation have **not** been observed; claims of operational success require Gate Evidence reports.